



DOUGLAS COLLEGE

COMMERCE AND BUSINESS ADMINISTRATION CSIS 3380: Full Stack Development with JavaScript

Assignment 2

(5 % towards your Final Grade)

Total Marks: 40

This assignment has two parts:

1. Client-Side: Use Fetch API and Axios to get data from an API and display it dynamically on a webpage.
2. Server-Side: Build a Node/Express server that also uses Fetch API or Axios to fetch the same API data and render it on a webpage using EJS templates.

Introduction:

Ajax is a web development technique that allows developers to make asynchronous HTTP requests without the need for a full-page reload. The traditional method for making AJAX requests was using the XMLHttpRequest interface, but the fetch() API and third-Party Libraries (e.g., Axios) are more suitable for modern web applications:

Using Fetch

The Fetch API provides an interface for retrieving resources. It works both in browsers and Node.js environment. The fetch function returns a Promise, which resolves once the request is completed. When we make an asynchronous request to a specific URL, it takes time for the request to travel across the internet and receive a response. Once the response is received, it contains JSON data within the body of the HTTP response. To extract and parse this data as JSON, we call response.json().

Using Axios

Axios is a Promise-based HTTP client for both the browser and Node.js. It is similar to the Fetch API and also offers additional powerful features that make working with HTTP requests more efficient and convenient. Like Fetch, Axios returns a JavaScript Promise, allowing for asynchronous request handling. However, Axios provides several advantages, including automatic JSON Parsing, simplified syntax and better error handling.

Using Async/Await

A popular approach for handling promises is to create an async function. Instead of waiting for the results of a promise to resolve and handling it with a chain of then functions, async functions can be told to wait for the promise to resolve before further executing any code found in the function. The await keyword is used before promise calls.

```
fetch("https://api.example.com/data")
  .then((response) => response.json()) // Convert response to JSON
  .then((data) => console.log(data)) // Handle data
  .catch((error) => console.error("Error:", error)); // Handle errors

async function fetchData() {
  try {
    const response = await fetch("https://api.example.com/data");
    const data = await response.json(); // Convert response to JSON
    console.log(data);
  } catch (error) {
    console.error("Error:", error);
  }
}

fetchData();
```

```
axios
  .get("https://api.example.com/data")
  .then((response) => console.log(response.data)) // Axios handles JSON automatically
  .catch((error) => console.error("Error:", error)); // Handle errors

async function fetchData() {
  try {
    const response = await axios.get("https://api.example.com/data");
    console.log(response.data); // No need to call .json()
  } catch (error) {
    console.error("Error:", error);
  }
}

fetchData();
```

Part 1 [20 marks]: Fetching and Displaying Images from the Unsplash API using Fetch and Axios (This is purely client side; there is no need to create a server for this part.)

Instructions:

Demonstrate the use of Fetch API and Axios to fetch data from a third-party API and dynamically update an HTML page. You are provided with an HTML template named **unsplashGallery.html**.

a. API to Use:

- Use Unsplash API: <https://unsplash.com/developers> to bring the desired images and image details on your web page
- You will need to register for a free API key (Access Key).
- The search should be based on a keyword entered by the user and the number of images to display. **Note:** You will have to structure the url:

``https://api.unsplash.com/search/photos?query=${query}&per_page=${count}&client_id=${accessKey}``

b. Create two separate scripts:

- fetchapi.js → uses Fetch API
- axios.js → uses Axios
- You must link both scripts to unsplashGallery.html but comment out one. I will uncomment it to test.
- You can use async/await or then/catch syntax.

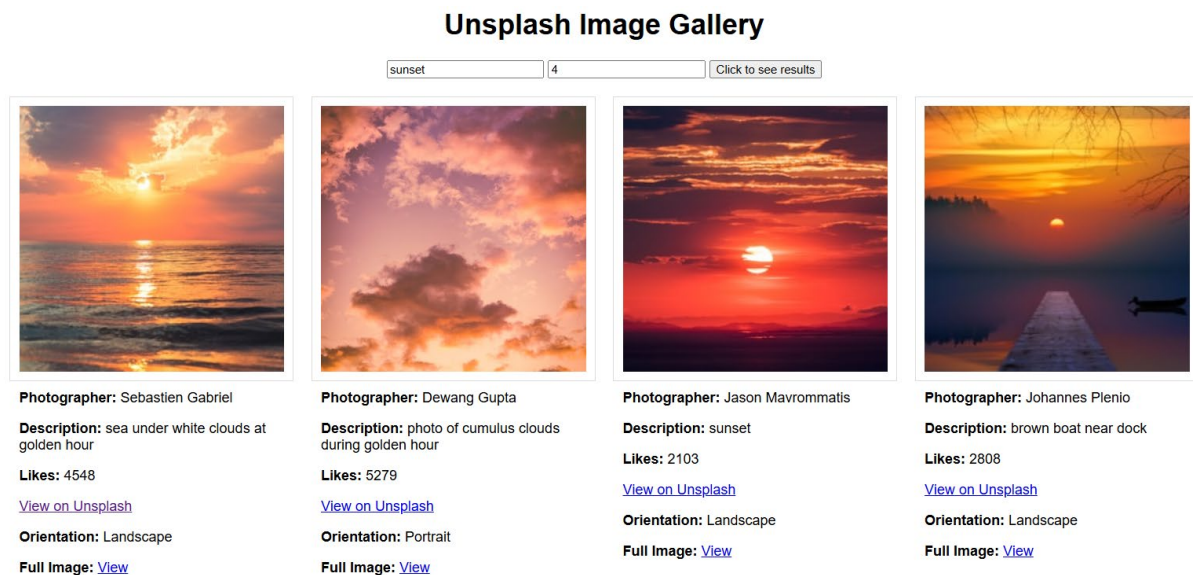
c. Functionality:

- The HTML page has a search box for the keyword and a number input for the number of images to display
- When the user enters the search keyword for the image and the number of images to be shown and clicks the button "Show Images",
 - Fetch the images from Unsplash API based on the keyword and number of images.
 - Display the images and relevant details for each image:
 - Photographer name
 - Image description (if available)
 - Number of likes
 - Image URL
 - Image type (orientation: portrait, landscape, square)
 - Small, medium, or full image URLs

d. Additional Requirements:

- Clear previous results before showing new ones.
- **Validation:** Show an alert if the keyword is empty or the number of images is less than 2.
- The webpage should look clean and visually appealing. Use your own styling for the displayed images and their details.

A sample webpage after the user enters the search keyword “sunset” and number of images “4”.



Part 2 [20 Marks]: In this part, you will build a server using Node/Express and render a webpage with the API data.

Complete the following tasks:

- a. [2 Marks] Handle a **GET** request from the client at the root route (localhost:3000/). When this route is accessed, send back the “*unsplashGallery.ejs*” file to the client as the response. (You will need to create this file yourself.) Your **EJS** page should include:
- A form with input fields for the **search keyword** and the **number of images** to display.
 - A **submit button** that initiates the image search when clicked.
 - Render this template in your GET route handler using the following syntax:
`res.render('unsplashGallery', { images: [] })`

Unsplash Image Gallery

Search Keyword:

Number of Images:

- b. [2 Marks] After the client fills in the required form fields (search keyword and number of images to display) and submits the form, handle the resulting **POST** request as follows:
- Define a POST route at **/search** to process the submitted form data.
 - Parse the input values from the request body.
 - Implement validation:
 - If the search keyword is empty or the number of images is less than 2, respond with a **400 (Bad Request)** status and an appropriate error message.
 - Otherwise, use the parsed input values to construct the Unsplash API request URL dynamically to fetch the corresponding image data from the **Unsplash API**.

- c. [8 Marks] Make a **GET** request to the external Unsplash API using both **Fetch API** and **Axios** (you may use either `async/await` or `.then()` syntax).
- In the **POST** route handler (`/search`), implement both the versions of the API call (to make a GET request to the Unsplash API.): one using **Fetch API** and one using **Axios**.
 - Comment out one implementation so that the other can be tested. (I will uncomment it to verify both versions.)
 - Retrieve and parse the image data returned by the Unsplash API dynamically, based on the user's search input and the specified number of images.
 - Send the resulting data to the **unsplashGallery.ejs** template for rendering.
 - Use `res.render('unsplashGallery', { images })` within the **POST** route handler to send the image data to the template.
- d. [8 Marks] Modify the existing **unsplashGaller.ejs** template to dynamically display the images fetched from the Unsplash API. The output should visually resemble the sample excerpt provided earlier.
- Ensure each image is displayed along with relevant details such as photographer name, image description, likes, **image URL**, image type and full image URL returned by the API.

Submission:

- You are provided with "**unsplashGallery.html**" that you will use to accomplish **Part1** tasks. For **Part2**, you will be creating your own "**unsplashGallery.ejs**" file.
- The assignment is to be completed individually. Any form of cheating or sharing of work may have serious consequences.
- Submit the compressed **AS2_YourFirstLastName** folder to the Blackboard. NO LATE SUBMISSION will be allowed.